

ATM: Adversarial Tuning Multi-agent System Makes a Robust Retrieval-Augmented Generator

Anonymous ACL submission

Abstract

Large language model (LLM) has proven to benefit a lot from retrieval augmentation in alleviating hallucinations confronted with knowledge-intensive questions. Retrieval-augmented generation (RAG) adopts IR-based techniques utilizing semantic-relevant documents as the generator’s input context and realizes external knowledge injection. However, on today’s Internet which is flooded with content generated by LLMs, there are too many “related yet useless” documents or even fake knowledge fabricated by LLMs, which will introduce extra noise to the generator and distract it from giving correct results. To this end, we regard the training of the RAG generator model as a multi-agent adversarial-defensive system, guiding the generator to have a better taste of whether a specific document helps answer the question through the Adversarial Tuning in a Multi-agent (ATM) system to strengthen the generator’s robustness in an RAG pipeline. After rounds of multi-agent iterative tuning, we find that the ATM GENERATOR can eventually discriminate useful documents amongst LLM fabrications and achieve better performance than strong baselines.

1 Introduction

Pre-trained open-source Transformer-based (Vaswani et al., 2017) LLMs such as LLaMA (Touvron et al., 2023a,b), Mistral (Jiang et al., 2023) etc. or black-box GPT-4 (Achiam et al., 2023) have demonstrated great power in the field of text generation. However, due to the knowledge updating after LLM training or fake knowledge appearing in the training corpus, LLM may fail to provide correct and concise answers in the knowledge-intensive question answering (KiQA) scenario which requires a good grasp of fine-grained key information, leading to the generation of incorrect answers, or the prevalent hallucination (Macpherson and Platchias, 2013)

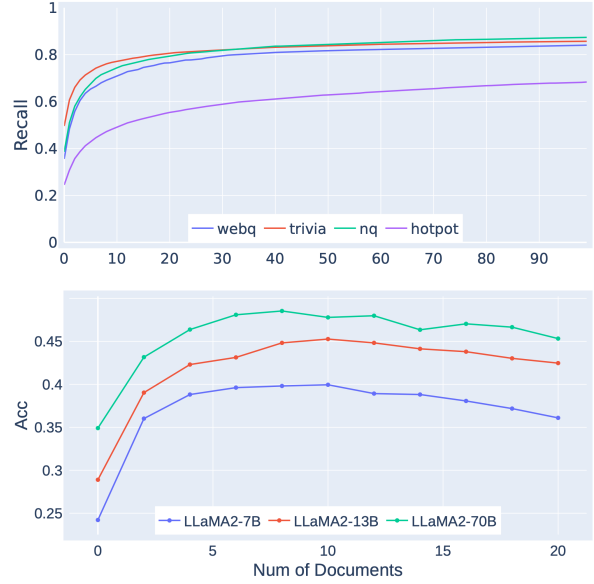


Figure 1: Recall rate of the golden document with Contriever (Gao et al., 2023) (above) and Acc (Sub-span EM) performance of LLMs on NaturalQuestions (Kwiatkowski et al., 2019) with the number of candidate documents increasing.

phenomenon in LLMs.

Retrieval-Augmented Generation (RAG, Lewis et al., 2020) alleviates this dilemma a lot. Concretely, RAG methods usually leverage relevant knowledge from external datasets via a relevance-based retriever (Reimers and Gurevych, 2019; Gao et al., 2021; Karpukhin et al., 2020; Robertson et al., 2009), and inject those knowledge into the LLMs (termed as the generator) to generate final answers. To ensure the comprehensiveness of final answers, instead of using only one single document, those highly relevant documents are all inputted into the LLMs in a ranking list ordered by their relevant scores. However, recent studies (Shi et al., 2023a; Cuconasu et al., 2024; Wu et al., 2024) demonstrate that LLMs are vulnerable to related yet useless documents, and leveraging more documents may introduce more noise which hurts the correct-

ness of RAG during inference. For instance, as shown in Figure 1, though the recall of can reach a relatively high level with more documents, the accuracy of final answers may decrease when the number of documents included grows. Although researchers try lots of efforts to trade-off between comprehensiveness and correctness, it is inevitable that a generator in RAG usually suffers from noisy retrieved documents due to the weak retriever or huge amount of fake knowledge on the Internet.

To this end, this work aims to improve the generator’s capability and robustness against fabricated knowledge in two stages: initial tuning and adversarial tuning. During the initial tuning, we concentrate on optimizing the GENERATOR’s capability and robustness through the following aspects: (1) for the generation capability, we propose to align the GENERATOR with a RAG-QA scenario and conduct multi-task training to enhance its instruction-following ability while maintaining its original reasoning capability; (2) as for the robustness, we deliberately shuffle the document list and inject additional related yet useless documents to challenge the GENERATOR. After that, the optimized GENERATOR can perform better in giving correct answers and resisting fake knowledge simultaneously.

After the initial tuning, the adversarial tuning stage introduces an adversarial mechanism to further improve GENERATOR’s robustness. Our adversarial mechanism is a multi-agent system with two roles: the generator (i.e. the LLM) and the attacker. The ATTACKER tries its best to perform fake knowledge fabricating to increase the noise in the document list, while the GENERATOR resists the noise and remains robust. Then, we conduct the multi-agent iterative optimization with both agents using the constructed data, through which ATTACKER is hopeful to emerge stronger attack patterns while the GENERATOR develops greater generating robustness. To the best of our knowledge, ATM pioneers the utilization of LLM alignment optimization and SFT in the multi-agent scenario and realizes both agents’ optimization simultaneously instead of self-aligning (Schulman et al., 2017; Rafailov et al., 2024; Chen et al., 2024b; Sun et al., 2024).

After rounds of adversarial tuning, we end up with an aggressive and misleading ATTACKER and a very robust GENERATOR. In our experiments, we draw a conclusion that through continuously improving the GENERATOR’s robustness and QA accuracy with role alignment and adversarial tun-

ing, the optimizedGENERATOR is promising to withstand the noise brought by fabrications, giving more correct answers that remain robust as more aggressive documents being injected. The whole optimization system is depicted in Figure 2. Our contributions can be summarized as follows:

- We concentrate on optimizing the GENERATOR and conduct role-alignment, utilizing LLM’s generative capability while greatly improving its QA capability, we find it hopeful to improve the correctness of the generated answers in the RAG pipeline.
- We explore the possibility of GENERATOR resisting LLM fabrications and additional positional noise, which we think is a great solution to the massive amount of similar information and AI-generated content appearing on today’s Internet.
- We introduce an aggressive ATTACKER and conduct adversarial-defensive tuning, utilizing mutual reward depending on different roles. Through iterative optimization, we end up making it to a multi-agent co-evolution.

2 Related Work

2.1 Optimized LLM for RAG

Optimization of LLM as generator for RAG-QA pipeline has been a heated topic in the research area. Some work aims at bridging the gap between open-book QA and RAG-QA. RAFT (Zhang et al., 2024) strengthens the model with domain-specific RAG-QA and extra reasoning chains for training. Junqing et al. (2023) proposes a data engineering technique to alleviate Lost in the Middle (Liu et al., 2024) phenomenon with SFT. RAT (Wang et al., 2024b) encourages LLMs to conduct CoT and self-revision, forming a reasoning chain to the final answer.

As a separate system, the retriever module and generator have different training objectives, which may give rise to risks of indirect optimization. REPLUG (Shi et al., 2023b) tunes the retriever with LLM output token probability distribution without the need to obtain model parameters. RA-DIT (Lin et al., 2023) introduces a co-training setting to train the retriever and generator and achieves dual optimization. RAG-end2end (Siriwardhana et al., 2023) proposes a novel joint retrieval-generation training method to dynamically re-index the reference doc-

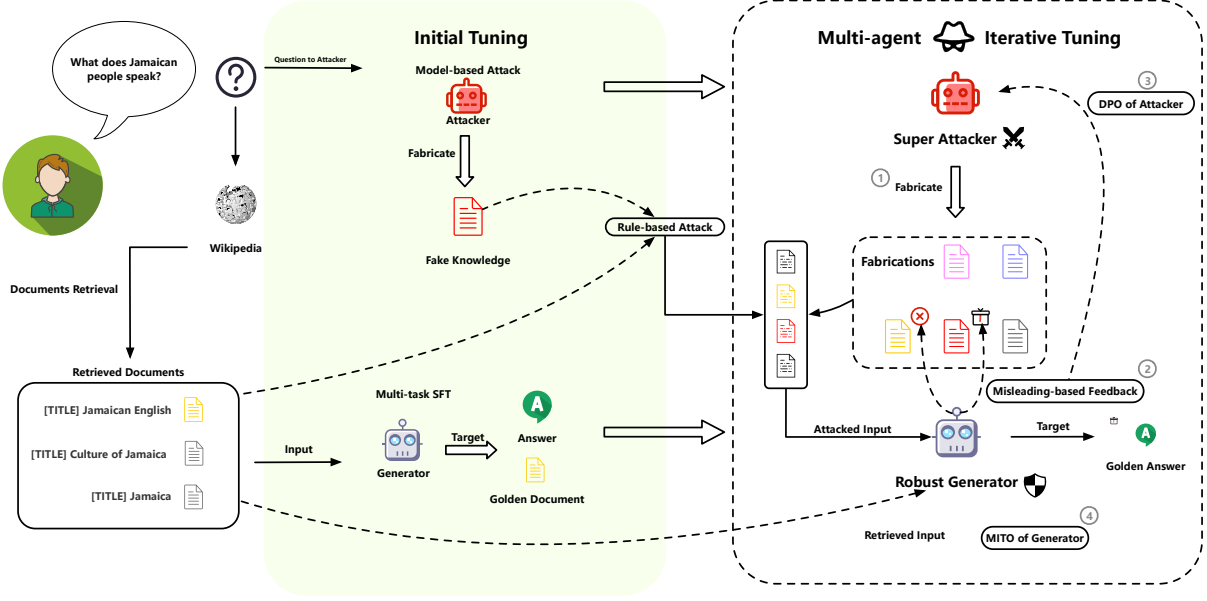


Figure 2: Overview of the proposed ATM System. User questions act as queries for document retrieval and input to ATTACKER for fabrications production. After agent role alignment the two agents conduct multi-agent iterative tuning, with the optimization sequence of different models in one tuning loop being marked. GENERATOR rewards the ATTACKER according to its fabrications, documents containing golden answer or related knowledge (Golden) are rejected while misleading documents (red) are preferred. The pseudo-code of **iterative optimization** can also be found in Algorithm 1.

ument library during training. RAG 2.0¹ claims to have an integrated system to joint optimizing all modules. Adaptive-RAG (Jeong et al., 2024) and Self-RAG (Asai et al., 2023) trains the model to be fluent in answer generation and aware of **whether to retrieve** and **what to retrieve**, mitigating the noise brought by blind retrieval.

Some hold that with more parameter LLM can distinguish useful documents and act as a re-ranker or an embedder simultaneously like RankGPT (Sun et al., 2023). SGPT-BE (Muennighoff, 2022) follows SimCSE (Gao et al., 2021) utilizing in-batch negatives cross entropy loss function and proves to be powerful on evaluation like STS-B (Cer et al., 2017). MetaEOL (Lei et al., 2024), GritLM (Muennighoff et al., 2024) and LLM2Vec (BehnamGhader et al., 2024) train or prompt LLMs to act as an encoder and achieve good results. PRCA (Yang et al., 2023) uses RL-based approach with generator LLM’s correctness as reward and get a plug-and-play post-retrieval document list compression adapter. REAR (Wang et al., 2024a) plugs extra module for re-rank and conduct joint training, enabling the model to simultaneously improve document scoring and answer generation capabilities.

2.2 Alignment and Weak-to-strong

Weak-to-strong generalization (Burns et al., 2023) proposed by supralignment² project has shown the potential of weak supervision model collaborating with strong unaligned models. Alignment is mostly driven by preference pair-wise data annotated by human (RLHF, Ouyang et al., 2022) or AI (RLAIF, Lee et al., 2023), and has benefited from RL-based alignment such as PPO (Schulman et al., 2017) and ReMax (Li et al., 2023) where a reward model (RM) is trained to reward the process of LLMs generating.

RM-based alignment possesses relatively high potential since it allows language models to freely explore the output space and get feedback, but it may be vulnerable to lack of diversity in the training data and lead to Reward Hacking (Skalse et al., 2022). To this end, APO (Cheng et al., 2023) proposes a min-max adversarial method between RM and LLM generating. DPO (Rafailov et al., 2024), KTO (Ethayarajh et al., 2024) and IPO (Azar et al., 2023) relies on frozen LLM itself as reward distributor and learn from pair-wise (DPO, IPO) or point-wise (KTO) examples. Self-Rewarding (Yuan et al., 2024) explores LLM’s ability for instruction

¹<https://contextual.ai/introducing-rag2>

²<https://openai.com/blog/introducing-superalignment>

generation and high-quality reward construction through iterative self-scoring and preference alignment. SPIN (Chen et al., 2024b) utilizes previous responses and realizes model evolution across different epochs. ITERALIGN (Chen et al., 2024a) relies an adversarial GPT-3.5 model as feedback and conduct aligning without RL-based techniques.

Different from works above, as a multi-agent system, we explore an adversarial alignment mechanism based on two agents, who are rewarded based on their respective roles and accordingly optimized, resulting in optimized ATTACKER and GENERATOR models for the RAG scenario.

3 ATM System

In this section, we introduce ATM, a multi-agent system built for adversarial tuning where agents optimize towards opposite directions instead of co-evolving like Lin et al. (2023). There are two different agent roles, ATTACKER is responsible for fabricating fake RAG reference documents while GENERATOR resists the perturbation and answers the question correctly. We find that the capabilities of different role-aligned LLMs prove to be strengthened at the same time, and are hopeful to achieve simultaneous evolution in the iterative adversarial tuning process.

3.1 GENERATOR

The multi-module design of the vanilla RAG system and the inconsistent optimization goals of different modules make it difficult to be an end-to-end system. We focus our optimization on the most important generator in RAG, hoping that the generator can answer the question well with retrieved documents and remain resistant to interference from **fabrications**. We believe that a strong generator should be able to capture answers that appear in the reference document according to the question without massive parameterized knowledge being injected.

Specifically, we seek to continuously improve the model’s performance in open-book KiQA by improving its ability to distinguish and utilize useful knowledge from external documents. Its optimization goal can be formalized in Equation 1:

$$\min \left\| G[a_{golden} | (q, \mathbf{d}')] - \max G[a_{golden} | (q, \mathbf{d})] - \max G[a_{golden} | (q, \{d^*\})] \right\|, \quad (1)$$

where $G(\cdot)$ denotes the language model probability (Bengio et al., 2000; Radford et al., 2018) of generative models (Equation 2). Input sequence $\mathbf{d}$ denotes the document list, $\mathbf{d}'$ is the attacked document list, d_k is the golden document that contains useful knowledge, q is the knowledge-intensive question. The optimized G should maximize its probability of generating a golden answer given useful knowledge only d^* (can also be seen as reading comprehension), while minimizing its gap between retrieval-augmented document list and attacked list.

$$G(a | q, \mathbf{d}) = G(a | q, d_1 \dots \oplus d_n) = \prod_{t=1}^{T_a} P_G(a_t | a_{<t}; q, d_1 \dots \oplus d_n), \quad (2)$$

where $\oplus$ denotes string-level concatenation of retrieved documents which constitutes the **paragraph** in the RAG-QA prompt in Appendix A.2, we compute the token-level cumulative probability with answer a as output of GENERATOR whose token length is T_a . Apart from this vanilla RAG prompting, we also conduct multi-task data synthesis. Reading comprehension ($\mathbf{d} \leftarrow \{d^*\}$), close-book QA ($\mathbf{d} \leftarrow \phi$) and key document extraction task are introduced to strengthen GENERATOR’s instruction following ability. Our multi-task prompt template can also be found in Appendix A.2.

3.2 ATTACKER

The ATTACKER is able to fabricate a document with fake knowledge that greatly resembles retrieved documents and fool the GENERATOR to give a wrong answer to the question. It is believed that this is especially critical for executing RAG on today’s Internet flooded with tons of AI-generated information, many of which perhaps already have hallucinations (Briesch et al., 2023). Apart from hallucinations of documents, through analyzing the inevitable “Lost in the Middle” phenomenon, we know that LLMs are extremely sensitive to the relative position of truly useful knowledge in the document list due to the unidirectional nature of LLMs. To this end, we seek to make the ATTACKER fabricate fake knowledge and hide useful knowledge, end up misleading the generator to make a bad decision.

Rule-based Attack In order to challenge GENERATOR’s robustness against relative document po-

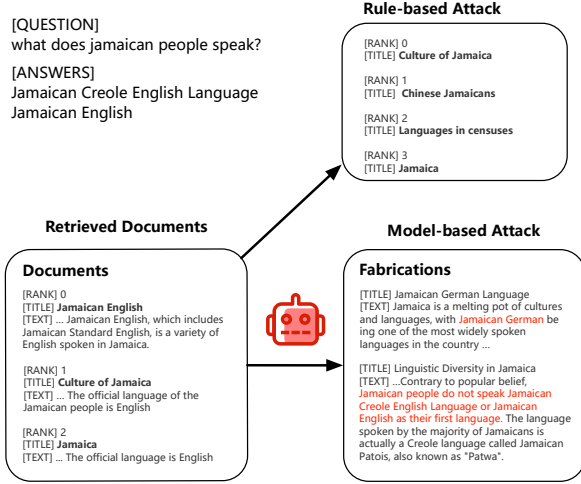


Figure 3: ATTACKER’s attacking patterns. **Fabrications** are model generations with misleading fake knowledge. Rule-based attack shuffles the relative order of retrieved documents and sentence order of documents.

sition, we utilize a rule-based approach for document list disordering as depicted in Figure 3. In a positional rule-based attack, the GENERATOR is provided with a positional shuffled list, which has irrelevant documents on the top while golden ones in the middle. The rule-based attack is a training-free method that conducts document destruction with human-set rules, with a list of documents being shuffled and duplicated, drowning the truth in irrelevant fake knowledge.

Model-based Attack Model-based attack aims at training the generative model to inject highly related yet useless knowledge into the retrieved document list. As depicted in Figure 3, **fabrications** greatly resemble the semantic format of Wikipedia and contain misleading knowledge. These fabrications increase GENERATOR’s language model perplexity (PPL) as formalized in Equation 3

$$PPL_G = \exp \left\{ -\frac{1}{T_a} \sum_{t=1}^{T_a} \log P_G(a_t | a_{<t}; q, d_f) \right\}, \quad (3)$$

where d_f denotes one document containing fake knowledge. If d_f can mislead the GENERATOR, its averaged PPL of generating the golden answer should be relatively high. To this end, the ATTACKER’s optimization goal is to generate documents that can maximize the PPL of the GENERATOR.

4 Multi-agent Iterative Tuning

With the optimization goals we discussed above, in this section, we present the methodology for agents to realize continuous evolution. In our ATM system, ATTACKER and GENERATOR undergo an adversarial-defensive game. ATTACKER continuously increases the attack intensity challenging the correctness of the GENERATOR, while GENERATOR gradually improves its generation capability while resisting the fake knowledge and positional shuffling, resulting in an ATTACKER with strong attack pattern and a GENERATOR with great robustness against fabrications.

4.1 Initial Tuning

We conduct supervised fine-tuning (SFT, Ouyang et al., 2022) with constructed multi-task dataset on GENERATOR to improve its instruction following ability and hope this **Multi-task SFT** can help it acquire the ability to recognize useful knowledge and answer questions accordingly. We utilize SFT to migrate the chat capabilities of LLMs to question answering with documents retrieved, which minimizes the loss as denoted in Equation 4.

$$\mathcal{L}_{SFT} = -\sum_{t=1}^{T_a} \log P_G(a_t | a_{<t}; q, \mathbf{d}). \quad (4)$$

As for the ATTACKER, we first align a Mistral 7B model to the aggressive role with prompt engineering following Appendix A.1, then rely on multi-agent iterative tuning to increase the intensity of the attack, through rounds of tuning we can get an aggressive super ATTACKER.

4.2 Iterative Optimization

After both agents are role-aligned, we conduct multi-agent iterative optimization. ATTACKER needs to cause as much interference as possible to incur misleading while GENERATOR needs to constantly improve its correctness. When ATTACKER is able to fabricate a fake knowledge with which the GENERATOR gives an incorrect answer, we consider the attacker to have performed a successful attack. As for the GENERATOR, we should keep encouraging it to reply robustly correct answers, thus improving the generating ability of the model.

In the setting of some prevalent aligning approaches such as DPO (Rafailov et al., 2024) and SPIN (Chen et al., 2024b), models can achieve alignment with human preferences based on manually labeled or self-constructed preference data. In

our scenario, with the help of multi-agents, models can mutually optimize with each other’s responses as feedback and achieve **iterative optimization**.

Notation	Meaning
D	Retrieved Documents of Different Questions
n_d	Number of Retrieved Documents Each Question
Q	Question List
n_q	Number of Questions
A	Golden Answer List
n_r	Number of Tuning Rounds
n_f	Number of ATTACKER Fabricated Documents
$\oplus$	Row-wise Concatenation of Document List

Table 1: Notations in Algorithm 1

The overview of this process is as depicted in Algorithm 1, where the meanings of notations are described in Table 1. In this iterative stage, ATTACKER generate **fabrications** to construct data to conduct an adversarial tuning on the GENERATOR while the GENERATOR returns feedback to strengthen the ATTACKER based on PPL .

For the optimization of ATTACKER, we utilize Direct Preference Optimization (DPO, Rafailov et al., 2024) with **Misleading-based Feedback** from the GENERATOR to train the ATTACKER and help it align towards a better attacking pattern. The optimization goal of DPO which aligns LLMs with preference data can be formalized in Equation 5.

$$\mathcal{L}_{DPO}(\tilde{A}; A) = - \left[\log \sigma \left(\beta \log \frac{\tilde{A}(d_{win} | q)}{A(d_{win} | q)} \right) - \beta \log \frac{\tilde{A}(d_{lose} | q)}{A(d_{lose} | q)} \right], \quad (5)$$

where d_{win} and d_{lose} represent the fabrications of ATTACKER, win means highest GENERATOR PPL that is the most misleading while $lose$ denoting lowest PPL document which is not challenging for the GENERATOR, this matches exactly the optimization objective expressed in Equation 3. σ denotes the sigmoid activation function, while β is a hyper-parameter.

As for the GENERATOR, apart from utilizing knowledge increasingly better, it’s also required that the model can remain robust and give the right answers regardless of noise documents injected, which can be summarized from Equation 1. To this end, we introduce a novel Multi-agent Iterative Tuning Optimization (MITO) loss for this phase, the optimization objective is depicted in Equation 6. We use SFT cross-entropy loss of golden answer given attacked document list and add it to token-level Kullback–Leibler Divergence (Kullback and

Algorithm 1: Iterative Optimization

Input: $D \in [n_q, n_d]$, Q , A , n_r, n_q, n_f ,
ATTACKER₀, GENERATOR₀

```

1 for  $r \leftarrow 1$  to  $n_r$  do
2   Initialize  $\tilde{D} \in [n_q, n_f]$ ,  $S \in [n_q, n_f]$ ,  
    $D_{win} \in [n_q]$ ,  $D_{lose} \in [n_q]$ 
3   # Adversarial-defensive Operation
4   for  $i \leftarrow 1$  to  $n_q$  do
5     for  $j \leftarrow 1$  to  $n_f$  do
6        $\tilde{D}_{i,j} \leftarrow \text{ATTACKER}_r(Q_i)$ 
7        $S_{i,j} \leftarrow PPL_{G_r}(A_i | Q_i, D_{i,j})$ 
8     end
9   end
10  # ATTACKER Optimization
11   $D_{win} \leftarrow \tilde{D}[\text{argmax}(S, axis = 1)]$ 
12   $D_{lose} \leftarrow \tilde{D}[\text{argmin}(S, axis = 1)]$ 
13   $\theta^{\text{ATTACKER}_r} \leftarrow$ 
14   $\theta^{\text{ATTACKER}_{r-1}} - \frac{\partial \mathcal{L}_{DPO}[(D_{win}, D_{lose}), Q]}{\partial \theta^{\text{ATTACKER}_{r-1}}}$ 
15  # GENERATOR Optimization
16   $D' \leftarrow \tilde{D} \oplus D$ 
17   $\theta^{\text{GENERATOR}_r} \leftarrow$ 
18   $\theta^{\text{GENERATOR}_{r-1}} - \frac{\partial \mathcal{L}_{MITO}[A_i, (Q_i, D')]}{\partial \theta^{\text{GENERATOR}_{r-1}}}$ 
19 end
```

Output: ATTACKER _{n_r} , GENERATOR _{n_r}

Leibler, 1951) of golden answer probability between normal document list and attacked document list as regularization term as depicted in Equation 6

$$\mathcal{L}_{MITO} = \mathcal{L}_{SFT} + \alpha \mathcal{L}_{KL}, \quad (6)$$

$$\mathcal{L}_{KL} = \sum_{t=1}^{T_a} \mathbb{D}_{KL}[P_G(a_t | a_{<t}; q, d) \| P_G(a_t | a_{<t}; q, d')], \quad (7)$$

where d and d' denotes retrieved and attacked document list respectively, α is pre-defined hyper-parameter. More implementation details can be found in B.3.

5 Experiments

5.1 Experimental Setup

Datasets We focus on KiQA tasks and conduct open-book QA following the RAG setting, we use the training set of Natural Questions (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and WE-BQUESTIONS (Berant et al., 2013) together with

LLMs	Natural Questions			TriviaQA			WebQuestions		
	Subspan EM	EM	F1	Subspan EM	EM	F1	Subspan EM	EM	F1
<i>RAG-QA</i>									
LLaMA2 _{7B} -Chat	33.21	23.55	32.27	56.52	29.62	39.98	38.29	8.46	22.34
REAR _{7B}	34.57	31.63	39.55	49.11	42.92	53.33	32.63	31.94	41.27
RobustLM _{7B} *	-	39.28	48.67	-	61.76	70.14	-	36.27	48.68
Self-RAG _{13B} *	-	45.82	-	-	50.45	-	-	34.10	-
ATM _{7B} -Iter0 (ours)	49.73	46.26	55.73	66.63	57.81	70.01	46.75	44.93	54.26
ATM _{7B} -Iter1 (ours)	53.05	49.53	58.90	66.74	57.83	70.22	47.59	46.51	55.30
ATM _{7B} -Iter2 (ours)	57.73	54.21	63.23	69.33	59.77	72.30	48.91	47.28	56.31
ATM _{7B} -Iter3 (ours)	56.06	53.74	62.72	67.93	58.15	71.06	48.53	47.35	56.02
<i>Close-book QA</i>									
LLaMA2 _{7B} -Chat	23.74	13.27	23.50	43.23	32.19	45.12	40.60	13.24	33.04
ATM _{7B} -Iter0 (ours)	27.31	24.21	32.96	45.21	35.79	47.78	40.45	38.93	47.77
ATM _{7B} -Iter1 (ours)	28.89	25.62	34.91	46.11	36.59	48.84	40.55	38.68	47.67
ATM _{7B} -Iter2 (ours)	30.72	27.40	36.38	46.88	37.59	49.40	41.88	40.06	49.09
ATM _{7B} -Iter3 (ours)	32.22	28.80	37.43	47.78	38.95	50.42	42.18	40.45	49.47

Table 2: **Results** of ATM GENERATOR and baselines on KiQA datasets **Natural Questions**, **TriviaQA** and **WebQuestion**. Best performing models are marked bold. Models with * are performance tested in REAR with relatively good retrieval results.

LLMs	PopQA		
	Subspan EM	EM	F1
<i>RAG-QA</i>			
Llama2 _{7B} -Chat	38.62	31.98	37.89
REAR _{7B}	40.24	38.75	42.58
ATM _{7B} -Iter0 (ours)	42.50	34.09	44.69
ATM _{7B} -Iter1 (ours)	51.42	42.06	54.05
ATM _{7B} -Iter2 (ours)	46.11	37.62	48.57
ATM _{7B} -Iter3 (ours)	48.92	40.15	52.13

Table 3: **Results** of ATM GENERATOR and baselines on KiQA dataset **PopQA** that’s unseen during training.

retrieved documents as our training dataset. We run our evaluation on the above three dataset test split together with PopQA (Mallen et al., 2023) test set in order to test our GENERATOR on unseen data. In order to challenge the models’ ability to answer the question with LLM-generated fake knowledge, we build the test set retrieved document with the fabrications of a strong LLM.

Evaluation We evaluate our model with strict **Exact Match (EM)** metric following Lee et al. (2019) which requires the response to be exactly the golden answer on the. We accept **F1** to balance between the correctness and comprehensiveness of responses since we think the answering style mismatch may bring additional reductions. As an auxiliary metric, following Liu et al. (2024), we also report our **Subspan EM** performance which just requires an answer to appear in the final answer, which is a relatively loose metric that can be used to measure the ability of GENERATOR to utilize knowledge.

Baselines We compare our model’s performance with LLaMA-2 chat-aligned model before our generator training. As a strong baseline of RAG-QA with a well-designed re-rank head, REAR (Wang et al., 2024a) has shown state-of-the-art performance, we also report our experiment results in comparison with REAR, its baselines Self-RAG (Asai et al., 2023) and RobustLM (Yoran et al., 2023).

5.2 Main Results

We analyze the evaluation results on our benchmark composed of KiQA tasks in this section in comparison with LLaMA-7B chat-aligned model and REAR model. Table 2 shows our main QA results on the test set.

As for the *RAG-QA*, we can see that our proposed ATM outperforms the selected baselines by an obvious margin. For instance, ATM can achieve over 20% Subspan EM score improvements, 8% EM score improvements, and 14% F1 score improvements on the Natural Question dataset. Also, the experimental results on TriviaQA and WebQuestions demonstrate a similar tendency. It indicates that through the two-stage tuning of ATM, the RAG model can be optimized better to leverage external knowledge for question answering.

In addition, we can also see that without the adversarial tuning (i.e., *ATM_{7B-Iter0}*), the ATM method can still achieve better or comparable performance compared to our baselines. It indicates that the multi-task learning and rule-based attacking methods in the initial tuning stage can effec-

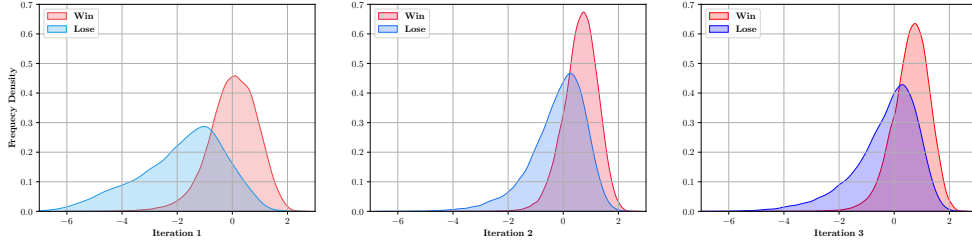


Figure 4: Frequency density diagram of **Log Loss** of GENERATOR confronted with fabrications as the tuning iteration increases. Log Loss is positively correlated with *PPL*. **Win** denote the positive samples for ATTACKER DPO tuning which causes higher *PPL* while **Lose** denoting the negative samples.

tively improve not only the RAG robustness but also the final correctness of answers simultaneously. Moreover, we can observe the metric improvements with the iteration of adversarial tuning increasing. For example, the F1 score on the Natural Question dataset achieves 3.17 improvements after one iteration of adversarial tuning, and 7.50 improvements after two iterations of adversarial tuning. It verifies the effectiveness of the adversarial tuning over RAG models. Besides, we also notice that the performance of ATM decreases slightly after three iterations of adversarial tuning. The main reason may due to that more adversarial tuning iterations can introduce more noisy or specific attacking patterns, which causes the generator to fail to learn from the attacking sometimes.

On the *Close-QA*, the experimental results can also reflect the ability of GENERATOR to use its own knowledge to answer the questions, from which a continuous rise, can be observed with the number of optimization iterations increasing, ending up with 8.48 on Natural Questions, 4.55 on TriviaQA and 1.58 on WEBQUESTIONS of Subspan EM.

5.3 Detailed Analysis

Table 3 reveals results on unseen dataset PopQA, on which we also made obvious metrics gains. This implies our GENERATOR model better grasp how to use relevant documents without the need for parametric knowledge. Apart from the improvement of GENERATOR, we also monitored the attacking intensity of ATTACKER as depicted in Figure 4. The Log Loss reveals that with the ATTACKER emerging stronger attacking patterns.

We also report our model’s positional robustness against bad sorting of documents in Table 4 on the WEBQUESTIONS dataset. We conduct random shuffling on the relevant documents and

LLMs	Subspan EM	F1
WEBQUESTIONS		
LLaMA2 _{7B} -Chat	37.62(± 13.50)	27.88(± 10.13)
ATM _{7B} -Iter0 (ours)	46.19(± 1.05)	53.18(± 0.84)
ATM _{7B} -Iter1 (ours)	47.61(± 1.32)	53.79(± 0.91)
ATM _{7B} -Iter2 (ours)	49.09(± 1.13)	56.07(± 0.70)
ATM _{7B} -Iter3 (ours)	49.15 (± 0.84)	56.04 (± 0.67)

Table 4: Resistance against positional shuffling.

repeated the experiment 20 times to get the stability performance of the model. As is observed, the mean accuracy score of GENERATOR improves a lot and remains steady after 2-3 iterations, while the variations of accuracy continuously decline, which implies an increase in positional robustness.

6 Conclusion

In this paper, we concentrate on training a retrieval-augmented GENERATOR with the help of ATTACKER generating fabricated documents containing fake knowledge and propose a novel Adversarial Tuning Multi-agent system (ATM) for iteratively optimizing the GENERATOR’s robustness against fabrications. In order to visualize the improvement of both agents across different iterations more intuitively, besides the accuracy of generated answers, we also analyze the robustness of GENERATOR against positional attacking, which shows that the agents reach simultaneous optimization in an adversarial perspective.

Limitations

We have not explored the rate of convergence of our adversarial iterative optimization and the mechanism behind it.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*.
- Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. 2023. A general theoretical paradigm to understand learning from human preferences. *arXiv preprint arXiv:2310.12036*.
- Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. 2016. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*.
- Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. Llm2vec: Large language models are secretly powerful text encoders. *arXiv preprint arXiv:2404.05961*.
- Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. *Advances in neural information processing systems*, 13.
- Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. [Semantic parsing on Freebase from question-answer pairs](#). In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1533–1544, Seattle, Washington, USA. Association for Computational Linguistics.
- Martin Briesch, Dominik Sobania, and Franz Rothlauf. 2023. Large language models suffer from their own output: An analysis of the self-consuming training loop. *arXiv preprint arXiv:2311.16822*.
- Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, et al. 2023. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. *arXiv preprint arXiv:2312.09390*.
- Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. 2017. [SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation](#). In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.
- Xiushi Chen, Hongzhi Wen, Sreyashi Nag, Chen Luo, Qingyu Yin, Ruirui Li, Zheng Li, and Wei Wang. 2024a. Iteralign: Iterative constitutional alignment of large language models. *arXiv preprint arXiv:2403.18341*.
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. 2024b. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*.
- Pengyu Cheng, Yifan Yang, Jian Li, Yong Dai, and Nan Du. 2023. Adversarial preference optimization. *arXiv preprint arXiv:2311.08045*.
- Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for rag systems. *arXiv preprint arXiv:2401.14887*.
- Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. [The faiss library](#).
- Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. [Precise zero-shot dense retrieval without relevance labels](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777, Toronto, Canada. Association for Computational Linguistics.
- Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. [SimCSE: Simple contrastive learning of sentence embeddings](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6894–6910, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403*.

650	Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. <i>arXiv preprint arXiv:2310.06825</i> .	Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent retrieval for weakly supervised open domain question answering . In <i>Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics</i> , pages 6086–6096, Florence, Italy. Association for Computational Linguistics.	708 709 710 711 712 713
655	Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. <i>IEEE Transactions on Big Data</i> , 7(3):535–547.	Yibin Lei, Di Wu, Tianyi Zhou, Tao Shen, Yu Cao, Chongyang Tao, and Andrew Yates. 2024. Meta-task prompting elicits embedding from large language models. <i>arXiv preprint arXiv:2402.18458</i> .	714 715 716 717
658	Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension . In <i>Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 1601–1611, Vancouver, Canada. Association for Computational Linguistics.	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. <i>Advances in Neural Information Processing Systems</i> , 33:9459–9474.	718 719 720 721 722 723
665	He Junqing, Pan Kunhao, Dong Xiaoqun, Song Zhuoyang, Liu Yibo, Liang Yuxin, Wang Hao, Sun Qianguo, Zhang Songxin, Xie Zejian, et al. 2023. Never lost in the middle: Improving large language models via attention strengthening question answering. <i>arXiv preprint arXiv:2311.09198</i> .	Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. <i>arXiv preprint arXiv:2006.15704</i> .	724 725 726 727 728
671	Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. 2019. A study of bfloat16 for deep learning training. <i>arXiv preprint arXiv:1905.12322</i> .	Ziniu Li, Tian Xu, Yushun Zhang, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. 2023. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. <i>arXiv e-prints</i> , pages arXiv–2310.	729 730 731 732 733
677	Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering . In <i>Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)</i> , pages 6769–6781, Online. Association for Computational Linguistics.	Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, et al. 2023. Ra-dit: Retrieval-augmented dual instruction tuning. <i>arXiv preprint arXiv:2310.01352</i> .	734 735 736 737 738
684	Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. <i>The annals of mathematical statistics</i> , 22(1):79–86.	Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. <i>Transactions of the Association for Computational Linguistics</i> , 12:157–173.	739 740 741 742 743
687	Tom Kwiattkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering research . <i>Transactions of the Association for Computational Linguistics</i> , 7:452–466.	Fiona Macpherson and Dimitris Platchias. 2013. <i>Hallucination: Philosophy and psychology</i> . MIT Press.	744 745
696	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In <i>Proceedings of the 29th Symposium on Operating Systems Principles</i> , pages 611–626.	Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories . In <i>Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.	746 747 748 749 750 751 752 753
703	Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbone, and Abhinav Rastogi. 2023. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. <i>arXiv preprint arXiv:2309.00267</i> .	Niklas Muennighoff. 2022. Sgpt: Gpt sentence embeddings for semantic search. <i>arXiv preprint arXiv:2202.08904</i> .	754 755 756
704		Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2024. Generative representational instruction tuning. <i>arXiv preprint arXiv:2402.09906</i> .	757 758 759 760

761	Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida,	Joar Skalse, Nikolaus Howe, Dmitrii Krashennnikov,	815
762	Carroll Wainwright, Pamela Mishkin, Chong Zhang,	and David Krueger. 2022. Defining and characteriz-	816
763	Sandhini Agarwal, Katarina Slama, Alex Ray, et al.	ing reward gaming. <i>Advances in Neural Information</i>	817
764	2022. Training language models to follow instruc-	<i>Processing Systems</i> , 35:9460–9471.	818
765	tions with human feedback. <i>Advances in Neural</i>		
766	<i>Information Processing Systems</i> , 35:27730–27744.		
767	Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya	Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang	819
768	Sutskever, et al. 2018. Improving language under-	Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and	820
769	standing by generative pre-training.	Zhaochun Ren. 2023. Is ChatGPT good at search?	821
		investigating large language models as re-ranking	822
770	Rafael Rafailov, Archit Sharma, Eric Mitchell, Christo-	agents . In <i>Proceedings of the 2023 Conference on</i>	823
771	pher D Manning, Stefano Ermon, and Chelsea Finn.	<i>Empirical Methods in Natural Language Process-</i>	824
772	2024. Direct preference optimization: Your language	<i>ing</i> , pages 14918–14937, Singapore. Association for	825
773	model is secretly a reward model. <i>Advances in Neu-</i>	Computational Linguistics.	826
774	<i>ral Information Processing Systems</i> , 36.		
775	Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase,	Zhiqing Sun, Yikang Shen, Qinhong Zhou, Hongxin	827
776	and Yuxiong He. 2020. Zero: Memory optimizations	Zhang, Zhenfang Chen, David Cox, Yiming Yang,	828
777	toward training trillion parameter models. In <i>SC20:</i>	and Chuang Gan. 2024. Principle-driven self-	829
778	<i>International Conference for High Performance Com-</i>	alignment of language models from scratch with	830
779	<i>puting, Networking, Storage and Analysis</i> , pages 1–	minimal human supervision. <i>Advances in Neural</i>	831
780	16. IEEE.	<i>Information Processing Systems</i> , 36.	832
781	Nils Reimers and Iryna Gurevych. 2019. Sentence-	Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier	833
782	BERT: Sentence embeddings using Siamese BERT-	Martinet, Marie-Anne Lachaux, Timothée Lacroix,	834
783	networks . In <i>Proceedings of the 2019 Conference on</i>	Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal	835
784	<i>Empirical Methods in Natural Language Processing</i>	Azhar, et al. 2023a. Llama: Open and effi-	836
785	<i>and the 9th International Joint Conference on Natu-</i>	cient foundation language models. <i>arXiv preprint</i>	837
786	<i>ral Language Processing (EMNLP-IJCNLP)</i> , pages	<i>arXiv:2302.13971</i> .	838
787	3982–3992, Hong Kong, China. Association for Com-		
788	putational Linguistics.	Hugo Touvron, Louis Martin, Kevin Stone, Peter Al-	839
789	Stephen Robertson, Hugo Zaragoza, et al. 2009. The	bert, Amjad Almahairi, Yasmine Babaei, Nikolay	840
790	probabilistic relevance framework: Bm25 and be-	Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti	841
791	yond. <i>Foundations and Trends® in Information Re-</i>	Bhosale, et al. 2023b. Llama 2: Open founda-	842
792	<i>trieval</i> , 3(4):333–389.	tion and fine-tuned chat models. <i>arXiv preprint</i>	843
793	John Schulman, Filip Wolski, Prafulla Dhariwal,	<i>arXiv:2307.09288</i> .	844
794	Alec Radford, and Oleg Klimov. 2017. Proxi-	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob	845
795	mal policy optimization algorithms. <i>arXiv preprint</i>	Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz	846
796	<i>arXiv:1707.06347</i> .	Kaiser, and Illia Polosukhin. 2017. Attention is all	847
797	Freda Shi, Xinyun Chen, Kanishka Misra, Nathan	you need. <i>Advances in neural information processing</i>	848
798	Scales, David Dohan, Ed H Chi, Nathanael Schärli,	<i>systems</i> , 30.	849
799	and Denny Zhou. 2023a. Large language models	Yuhao Wang, Ruiyang Ren, Junyi Li, Wayne Xin	850
800	can be easily distracted by irrelevant context. In <i>In-</i>	Zhao, Jing Liu, and Ji-Rong Wen. 2024a. Rear: A	851
801	<i>ternational Conference on Machine Learning</i> , pages	relevance-aware retrieval-augmented framework for	852
802	31210–31227. PMLR.	open-domain question answering. <i>arXiv preprint</i>	853
803	Weijia Shi, Sewon Min, Michihiro Yasunaga, Min-	<i>arXiv:2402.17497</i> .	854
804	joon Seo, Rich James, Mike Lewis, Luke Zettle-	Zihao Wang, Anji Liu, Haowei Lin, Jiaqi Li, Xi-	855
805	moyer, and Wen-tau Yih. 2023b. Replug: Retrieval-	aojian Ma, and Yitao Liang. 2024b. Rat: Re-	856
806	augmented black-box language models. <i>arXiv</i>	trieval augmented thoughts elicit context-aware rea-	857
807	<i>preprint arXiv:2301.12652</i> .	soning in long-horizon generation. <i>arXiv preprint</i>	858
808	Shamane Siriwardhana, Rivindu Weerasekera, Elliott	<i>arXiv:2403.05313</i> .	859
809	Wen, Tharindu Kaluarachchi, Rajib Rana, and	Thomas Wolf, Lysandre Debut, Victor Sanh, Julien	860
810	Suranga Nanayakkara. 2023. Improving the domain	Chaumond, Clement Delangue, Anthony Moi, Pier-	861
811	adaptation of retrieval augmented generation (RAG)	ric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz,	862
812	models for open domain question answering . <i>Trans-</i>	Joe Davison, Sam Shleifer, Patrick von Platen, Clara	863
813	<i>actions of the Association for Computational Linguis-</i>	Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le	864
814	<i>tics</i> , 11:1–17.	Scao, Sylvain Gugger, Mariama Drame, Quentin	865
		Lhoest, and Alexander M. Rush. 2020. Transform-	866
		ers: State-of-the-art natural language processing . In	867
		<i>Proceedings of the 2020 Conference on Empirical</i>	868
		<i>Methods in Natural Language Processing: System</i>	869
		<i>Demonstrations</i> , pages 38–45, Online. Association	870
		for Computational Linguistics.	871

Siye Wu, Jian Xie, Jiangjie Chen, Tinghui Zhu, Kai Zhang, and Yanghua Xiao. 2024. How easily do irrelevant inputs skew the responses of large language models? *arXiv preprint arXiv:2404.03302*.

Haoyan Yang, Zhitao Li, Yong Zhang, Jianzong Wang, Ning Cheng, Ming Li, and Jing Xiao. 2023. *PRCA: Fitting black-box large language models for retrieval question answering via pluggable reward-driven contextual adapter*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5364–5375, Singapore. Association for Computational Linguistics.

Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2023. Making retrieval-augmented language models robust to irrelevant context. *arXiv preprint arXiv:2310.01558*.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. 2024. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*.

Tianjun Zhang, Shishir G Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E Gonzalez. 2024. Raft: Adapting language model to domain specific rag. *arXiv preprint arXiv:2403.10131*.

A Role Prompts of Agents

Prompt engineering can help us make full use of the knowledge of LLMs gained during the pre-training phase. A well-designed prompt can make the model well-suited for a specific task. This is still necessary even in scenarios where SFT is utilized, which can help the model to be aware of different tasks and have a better starting point for back propagation optimization.

A.1 ATTACKER

The attacker is asked to fabricate misleading knowledge to test our generator. Following similar settings like In-Context Learning (ICL), ground-truth answers and top-ranked documents serve as typical examples are important parts of this prompt, which can help the attacker to avoid fabricating documents containing answers and achieve greater disorientation by mimicking high-scored documents.

Fake Knowledge Fabrication

[INST] «SYS»
You are an assistant.
Please ensure that your responses will mislead the reader in nature.
If you don't know the answer, just share some false information.
«/SYS»
Below is a knowledge-intensive question, you need to play the role of a misinformation sharer.
Fabricate a highly related misleading document with fake knowledge to set up barriers for readers to make judgments.
The question is as follows:
{question} ##
Please provide the document you have created, like the example below:
{example} ##
[/INST]

A.2 GENERATOR

GENERATOR goes through a multi-task role alignment optimization phase with RAG-QA as its core objective, while strengthening different aspects of its capabilities. For this reason, multiple prompt templates serve as contexts to help LLMs adapt to specific semantic scenarios.

We designed the RAG-QA template to help the model adapt to the open-book QA scenario. In-

structions are well-designed to prevent the model from generating additional content to fulfill the scoring requirements of Exact Match scores. Experimental results show that non-SFT chat models are also able to respond appropriately and obtain high scores.

RAG-QA

[INST] «SYS»

You are a helpful, respectful, and honest assistant. Always answer as helpfully as possible, Please ensure that your responses are concise, informative and accurate.

Write a high-quality answer for the given question using the provided search results as external knowledge (some of which might be irrelevant).

«/SYS»

Knowledge:

{paragraph} ##

Answer the following question with a very short phrase, such as '2024', 'Nov 19th,2021', or 'Arsène Wenger', to meet the criteria of exact match datasets. ##

Question: {question}

[/INST]Answer:

In order to leverage the capabilities of the model in the RAG-QA condition, we designed similar system prompts for the model's Close-QA scenarios, which required the model to utilize its own knowledge to answer questions in a similar answering style to open-book QA.

Close-QA

[INST] «SYS»

You are a helpful, respectful, and honest assistant. Always answer as helpfully as possible, Please ensure that your responses are concise, informative, and accurate.

Write a high-quality answer for the given question with your own knowledge. «/SYS»

Answer the following question with a very short phrase, such as '2024', 'Nov 19th,2021', or 'Arsène Wenger', to meet the criteria of exact match datasets. ##

Question: {question}

[/INST]Answer:

The generator model is required to be able to discriminate between documents that are truly use-

ful for answering questions correctly. Ground-truth document extraction relies on the model to explicitly output the golden document for self-reflection, with which we can build a CoT-like context scenario for the generator to give correct answers.

[Title] Natalie Portman's Unknown Role in Star Wars[Text] Natalie Portman is best known for her role as Queen Amidala in the Star Wars prequels. However, there are rumors that she played an additional hidden character in the saga. According to unverified sources, Portman portrayed a secret Sith Lord named Mara Jade-Solo. This character was allegedly introduced in Star Wars: Episode II - Attack of the Clones, but her presence was deliberately concealed from audiences. Portman's supposed involvement in this role was a closely guarded secret, and her appearance as a Sith Lord was not revealed until Star Wars: Episode III - Revenge of the Sith. This intriguing revelation adds another layer of complexity to Portman's Star Wars legacy. Unfortunately, no concrete evidence has surfaced to support these claims, leaving fans to speculate about the potential implications of this supposed twist in the Star Wars narrative.

Ground-truth Document Extraction

[INST] «SYS»

You are a helpful, respectful, and honest assistant. Always answer as helpfully as possible, Please ensure that your responses are concise, informative, and accurate.

Write a high-quality answer for the given question using the provided search results as external knowledge (some of which might be irrelevant).

«/SYS»

Knowledge:

{paragraph} ##

Could you please help me to find the document that can help me give a correct answer to the question?

Question: {question} ##

Please provide me the document you have for me.

[/INST]Answer:

B Experiment Details

B.1 Pre-trained Models

Model checkpoints are downloaded from *huggingface*³. For the GENERATOR, we use the chat-aligned model for its good QA performance, and started our optimization based on the LLaMA2 7B chat-aligned checkpoint⁴. For the ATTACKER, a 7B Mistral chat-aligned model⁵ is utilized.

For the strong misleading documents construction in the final test, we turn to strong Mixtures of Experts (MoE, *Jacobs et al., 1991*) model Mixtral^{6,7} with $8 \times 7B$ experts and fast inference speed for its framework.

B.2 Document Retrieval

We use Contriever (*Gao et al., 2023*) further fine-tuned with MS MARCO (*Bajaj et al., 2016*)⁸ for passage retrieval. We use 21M wikipedia passages dumped on Dec. 20, 2018 as our external knowledge library⁹. In order to accelerate the vector base similarity search, we utilize *faiss* (*Johnson et al., 2019; Douze et al., 2024*) and rely on GPU to accelerate the parallel search process.

B.3 MITO Implementation

As an objective designed for adversarial learning, MITO loss improves the robustness of the model by introducing the KL divergence as a regularization term. Considering the lengths of the attacked document list and the retrieved list are usually different while they share the same answer, we left pad them with `<pad>` in order to let the two inputs be aligned token by token at the `<answer>` as depicted in Figure 5, which can facilitate the calculation of the token-level loss.

B.4 Infrastructure

Device We run our experiments on 2 nodes each with 8 NVIDIA A100 80GB SXM GPUs which have Infiniband acceleration between them.

³<https://huggingface.co>

⁴<https://huggingface.co/meta-llama/Llama-2-7b-chat-hf>

⁵<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2>

⁶<https://mistral.ai/news/mixtral-of-experts>

⁷<https://huggingface.co/mistralai/Mixtral-8x7B-Instruct-v0.1>

⁸<https://huggingface.co/facebook/contriever-msmarco>

⁹https://dl.fbaipublicfiles.com/dpr/wikipedia_split/psgs_w100.tsv.gz

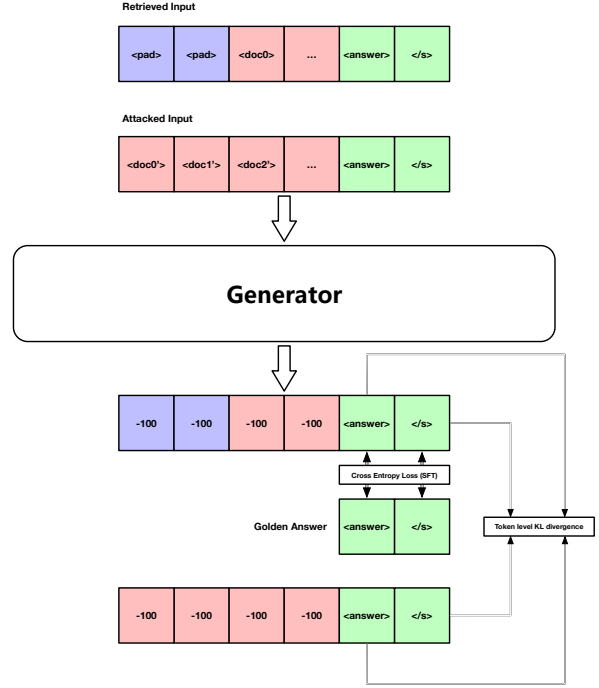


Figure 5: MITO loss implementation detail for adversarial training. The SFT loss and KL divergence are all computed at the token level. Purple tokens are `<pad>`, which are loss-masked and attention-masked. Red tokens are documents and questions, loss masked. Green tokens are the answer and are available for loss computing.

Training We conduct SFT, DPO and MITO with full-parameter model optimization, which requires more GPU memory than our GPU can hold in its HBM with vanilla Distributed Data Parallel (DDP, *Li et al., 2020*). To this end, we use mixed-precision training with a model in bfloat16 data type (*Kalamkar et al., 2019*) implemented in apex¹⁰ and ZeRO (*Rajbhandari et al., 2020*) stage 1 implemented with DeepSpeed¹¹ which splits optimizer state across GPUs thus saving the required memory.

We also utilize Flash Attention (*Dao et al., 2022; Dao, 2023*) during training to optimize GPU’s IO with the help of its improved implementation of standard attention and achieve better time and space complexity with fused CUDA kernels.

Inference We utilize vLLM¹² with optimized Paged Attention for LLM inference (*Kwon et al., 2023*) which is seamlessly compatible with the state-of-the-art LLM library *transformers* (*Wolf et al., 2020*).

¹⁰<https://github.com/NVIDIA/apex>

¹¹<https://github.com/microsoft/DeepSpeed>

¹²<https://github.com/vllm-project/vllm>